

MODULE 4

GitHub turns a group into a team

Imagine four students working on one project. Everyone edits the same file—and nobody knows whose version is correct.

Dr. R. Varalakshmi

**Dean of Academics, Associate Professor and Head
M.Sc. Data Science and Artificial Intelligence**

“WHO CHANGED MY FILE?”

**GitHub gives the team a shared workspace with conversations, tasks, review
and controlled changes.**

**METAPHOR: collaboration is a group project with a shared notebook and a teacher-approved
review process.**

Repository Sharing & Collaborators

First, decide who can work with the repository and what each person should be allowed to do.

REPOSITORY SHARING

Share the project with the people who need access.

Think: “Who needs the key?”

COLLABORATORS

People with access can contribute according to the permissions available to them.

Think: “Who is on the project team?”

Classroom example: Teacher owns the repository → student team members collaborate → everyone follows the project rules.

Issues & Discussions

Use Issues to track work and problems. Use Discussions for questions, ideas and community conversation.

ISSUES

“The login page has a bug.”

Issue can record:

- Problem
- Task
- Assignee
- Labels
- Progress

DISCUSSIONS

“Which design should we use?”

Useful for:

- Questions
- Ideas
- Announcements
- Community conversation

Shortcut: Issue = “something to track.” Discussion = “something to talk about.”

Pull Requests, Forking & Branches

These concepts help teams work on changes without immediately disturbing the main project.

BRANCH

A separate line of work.

Example:
main
↳ feature-login

PULL REQUEST

A request to review and
merge proposed changes into
another branch.

FORK

A copy of another repository
under your account/space,
commonly used to work
independently and contribute
back.

Metaphor: Branch = practice room • Pull Request = “Teacher, please review this” • Fork = photocopy your own working copy.

GitHub Notifications

Notifications help team members notice activity that needs attention.

Mentions

Someone asks you directly.

Issues

A task or problem changes.

Pull Requests

A review or update needs attention.

Discussions

A conversation has new activity.

Best practice: notifications are signals, not noise. Check what needs your action and respond clearly.

0 Repository Sharing &
1 Collaborators

0 Issues & Discussions
2

0 Pull Requests
3

0 Forking & Branches
4

0 Notifications
5

0 **Projects, Labels &
6 Milestones**

0 Comments, Suggestions
7 & Reviews

0 Naming &
8 Documentation Standards

0 File Organization &
9 Backup

1 Open Source, Copyright
0 & Academic Integrity

6 HOURS
TOTAL MODULE

Projects, Labels & Milestones

Turn a collection of tasks into a visible plan.

PROJECTS

Organize and track work
across a project.

LABELS

Tag work: bug, feature,
documentation, urgent.

MILESTONES

Group issues toward a
meaningful goal or deadline.

Example: Milestone “Version 1” → Labels “Bug / Feature / Docs” → Project board tracks progress.

0 Repository Sharing &
1 Collaborators

0 Issues & Discussions
2

0 Pull Requests
3

0 Forking & Branches
4

0 Notifications
5

0 Projects, Labels &
6 Milestones

0 **Comments, Suggestions
7 & Reviews**

0 Naming &
8 Documentation Standards

0 File Organization &
9 Backup

1 Open Source, Copyright
0 & Academic Integrity

Comments, Suggestions & Reviews

Good collaboration is not only about changing files—it is about explaining and improving changes.

COMMENT

Ask a question or explain something near the work.

SUGGESTION

Propose a specific improvement instead of silently changing someone's work.

REVIEW

Evaluate a proposed change and provide useful feedback.

Review rule: criticize the work, not the person. Be specific, respectful and evidence-based.

Naming & Documentation Standards

A collaborative repository should be understandable even when the original author is not available.

NAMING CONVENTIONS

Use meaningful names.

- ✓ attendance-analysis
- ✓ monthly-report.md

- X abc123
- X newnewfinal2

DOCUMENTATION STANDARDS

Explain:

- Purpose
- Structure
- How to use
- Important decisions
- Contribution rules

Metaphor: naming conventions are road signs. They help everyone find the right destination.

0 1	Repository Sharing & Collaborators
0 2	Issues & Discussions
0 3	Pull Requests
0 4	Forking & Branches
0 5	Notifications
0 6	Projects, Labels & Milestones
0 7	Comments, Suggestions & Reviews
0 8	Naming & Documentation Standards
0 9	File Organization & Backup
1 0	Open Source, Copyright & Academic Integrity

File Organization & Backup Strategy

Collaboration becomes safer when the project is organized and valuable work has recovery options.

FILE ORGANIZATION

Keep related files together.
Use predictable folders.
Avoid duplicate “final” files.
Keep temporary files out when appropriate.

BACKUP STRATEGY

Repository history helps recover previous states, but teams should still understand what data needs independent backup and where sensitive information belongs.

Safety rule: GitHub is not permission to upload passwords, API keys or private student information.

Open Source, Copyright, Licensing & Academic Integrity

Great collaboration also means respecting ownership, permission and honest academic work.

OPEN SOURCE

Use and contribute according to the project's license.

COPYRIGHT

Do not assume everything online is free to copy.

LICENSING

A license explains what others may do with the work.

ACADEMIC INTEGRITY

Give credit. Do your own work. Disclose meaningful assistance.

YOUR TEAM CHALLENGE → CREATE • DISCUSS • REVIEW • IMPROVE • GIVE CREDIT



MODULE 4 - COLLABORATION USING GITHUB

Work Together. Build Better. Ship Faster.

Collaboration turns "Me" into "We"



1. WORKING WITH OTHERS

Collaboration starts with sharing and inviting others to contribute.

Repository Sharing

- Share your repository with specific people or teams.

Collaborators

- Invite collaborators.
- Assign roles:

Role	Can
Read	View code
Write	Add/modify code
Admin	Manage settings, add collaborators

2. ISSUES & DISCUSSIONS

Plan, discuss and track work in a shared space.

Issues

- Track tasks, bugs, enhancements.
- Assign to team members.
- Add labels, due dates.

New Issue

Title: Bug in login page

Assignees: @aaron

Labels: bug, high priority

Discussions

- Ask questions.
- Share ideas.
- Have open conversations with the community.

Discussions

Q: How to connect API?

A: Here is the solution...

3 likes, 2 comments

PULL REQUESTS (CONCEPT)

Propose changes, review and merge safely into the main project.

How it works

- Create a branch
- Make changes
- Open Pull Request
- Review & discuss
- Merge when approved

Pull Requests keep the main code stable and quality high.

FORKING & BRANCHES (CONCEPT)

Forking

Create your own copy of a repository to experiment freely.

Branches

Work on different features without affecting the main project.

5. GITHUB NOTIFICATIONS

Stay updated with activities that matter to you.

- Mentions (@yourname)
- Issues assigned
- Pull request updates
- Comments
- Release & workflow alerts

Notifications

- @aaron mentioned you in #25
- Pull request #12 was reviewed
- Issue #8 was assigned to you
- New comment on README.md

6. GITHUB PROJECTS, LABELS & MILESTONES

GitHub Projects

Organize tasks in boards (Kanban style).

To Do: Task 1, Task 2

In Progress: Task 3

Done: Task 4 ✓

Labels

Categorize issues and pull requests.

bug, enhancement, documentation, help wanted

Milestones

Group issues into milestones and track progress.

60%

7. DOCUMENT REVIEW: COMMENTS, SUGGESTIONS & REVIEWS

Comments

- Add comments on code, issues or PRs.

app.py

```
10 def add(a, b):
11-   return a+b
11+   return a + b
```

@reviewer Consider adding validation here.

Reviews

- Approve changes.
- Request changes.
- Comment and discuss.

Approve, Request changes, Comment

Suggestions

- Suggest improvements directly.

Suggested change

return a + b → return a+b

Add suggestion

★ BEST PRACTICES

Naming Conventions

Use clear, consistent names for repos, branches, files, and issues.

Documentation Standards

Write clear README, comments, and docs.

File Organization

Follow a clean and logical folder structure.

Backup Strategy

Regularly backup important data. Use releases or mirrors.

Example Folder Structure

```
my-project/
├── docs/
├── src/
│   ├── main.py
│   └── utils.py
├── tests/
├── assets/
└── README.md
```

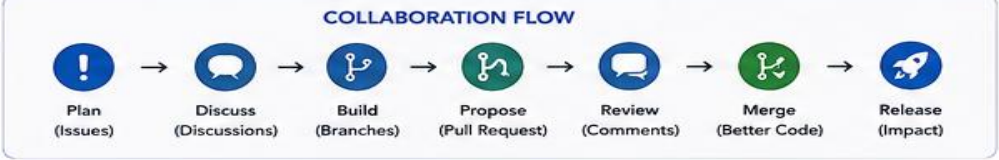
Good habits today = Healthy projects tomorrow! 😊

ETHICS & RESPONSIBILITY

- Open Source**: Share knowledge and code for the benefit of others.
- Copyright**: Respect others' work. Do not copy without permission.
- Licensing**: Use proper licenses (MIT, Apache 2.0, GPL, etc.) to define how your code can be used.
- Academic Integrity**: Give credit, avoid plagiarism, and be honest in all your work.

KEY TAKEAWAYS

- Collaborate respectfully.
- Communicate clearly.
- Review before merging.
- Use tools to stay organized.
- Follow ethical and best practices.
- Together, we build amazing things!



CALL TO ACTION

Start collaborating today on GitHub! Share. Discuss. Review. Improve.

Build together and create something awesome!